Lab Assignment 15.2

Program	**: B. Tech (CSE)**
Course Title	**: AI Assisted Coding**
Course Code	**: 23CS002PC304**
Semester	**: III**
Academic Session	**: 2025-2026**
Name of Student	**: Sruthi**
Enrollment No.	**: 2403A51L10**
Batch No.	**: 51**
Date	**: 17-03-2026**

Task 1:

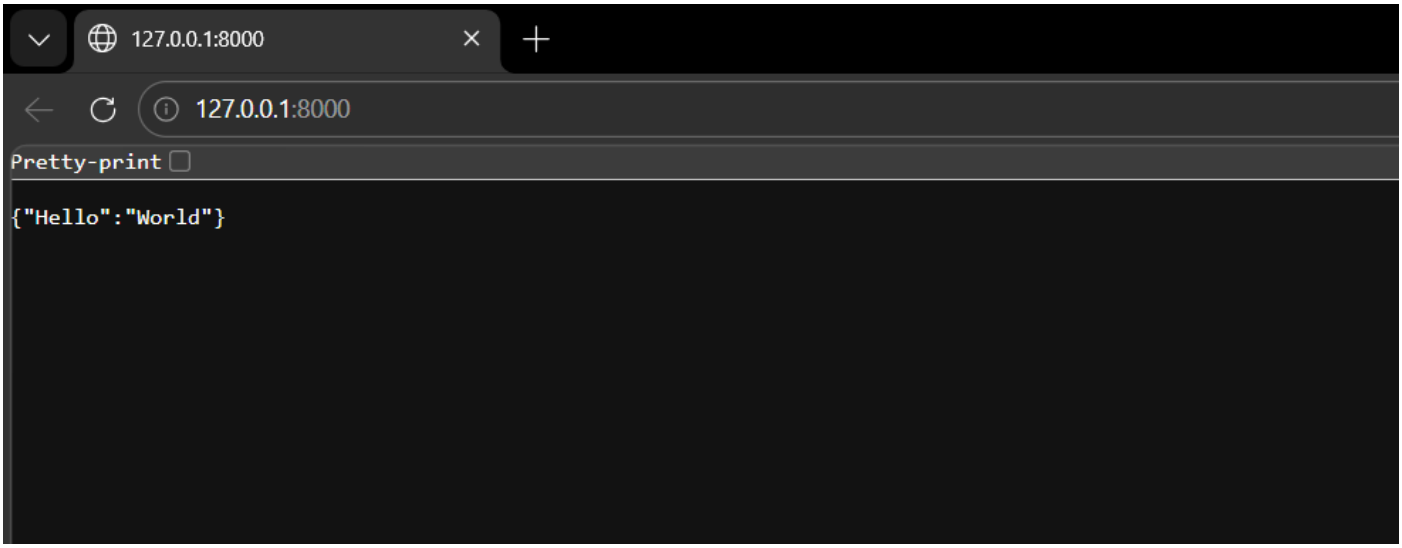
Prompt: Generate a basic Fast API application with one endpoint

Code:

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"Hello": "World"}
```

Output:

```
(.venv) PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> python -m uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\BURRA SRUTHI\\OneDrive\\Desktop\\3_2\\AI\\AI_code']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [41568] using StatReload
INFO:     Started server process [12964]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```



Explanation:

This code creates a basic FastAPI application with one endpoint. The endpoint is defined using the @app.get ("/") decorator, which means it will respond to GET requests at the root URL ("/"). When a request is made to this endpoint, the read_root function is called, which returns a JSON response with the message "Hello": "World".

Task 2:

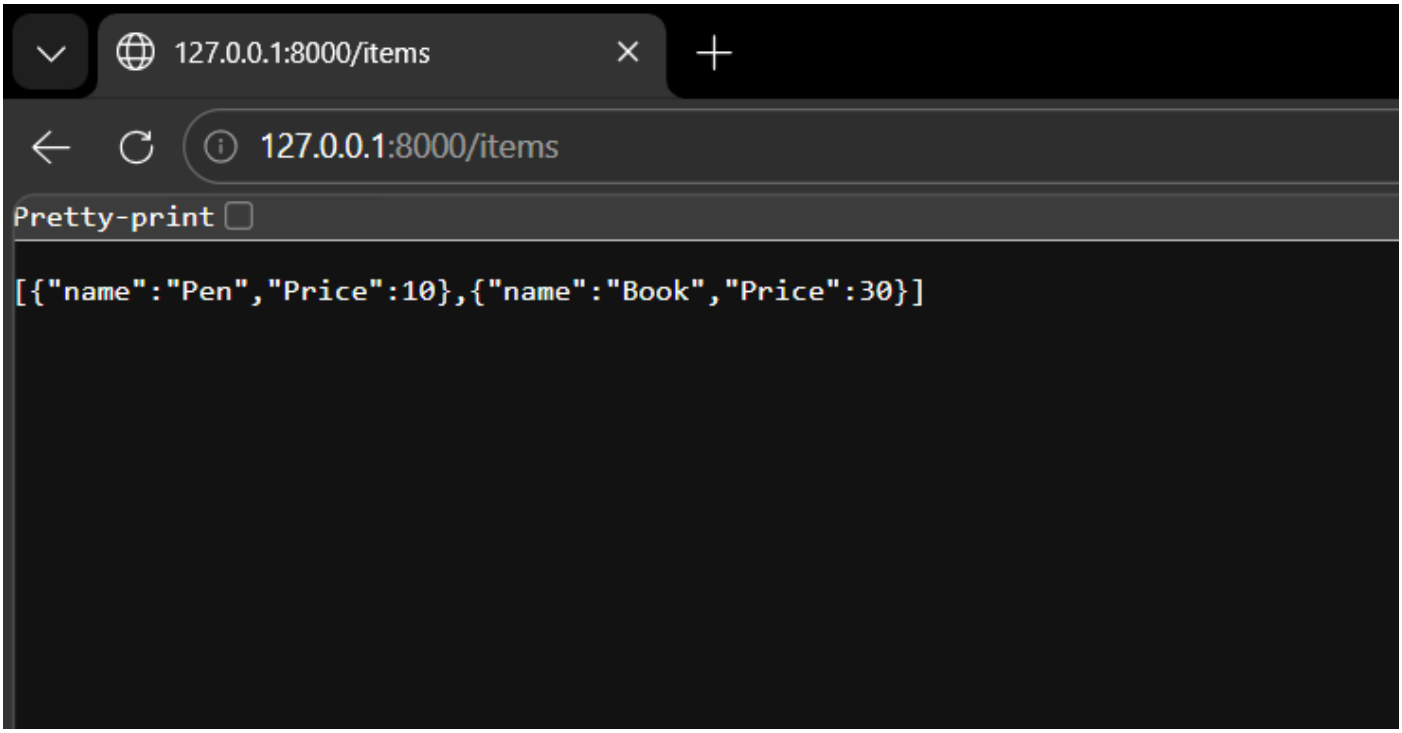
Prompt: Use a python list as temporary storage and Implement endpoints to add and retrieve products.

Code:

```python
from fastapi import FastAPI
app = FastAPI()
items = []
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

Output:

```
(.venv) PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> python -m uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\BURRA SRUTHI\\OneDrive\\Desktop\\3_2\\AI\\AI_code']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [31776] using StatReload
INFO:     Started server process [33664]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```



Explanation:

This code creates a FastAPI application with two endpoints. The first endpoint (GET /items) retrieves the list of items stored in the 'items' list, while the second endpoint (POST /items) allows you to add a new item to the list by sending a JSON object in the request body. The added item is then returned in the response along with a success message.

Task 3:

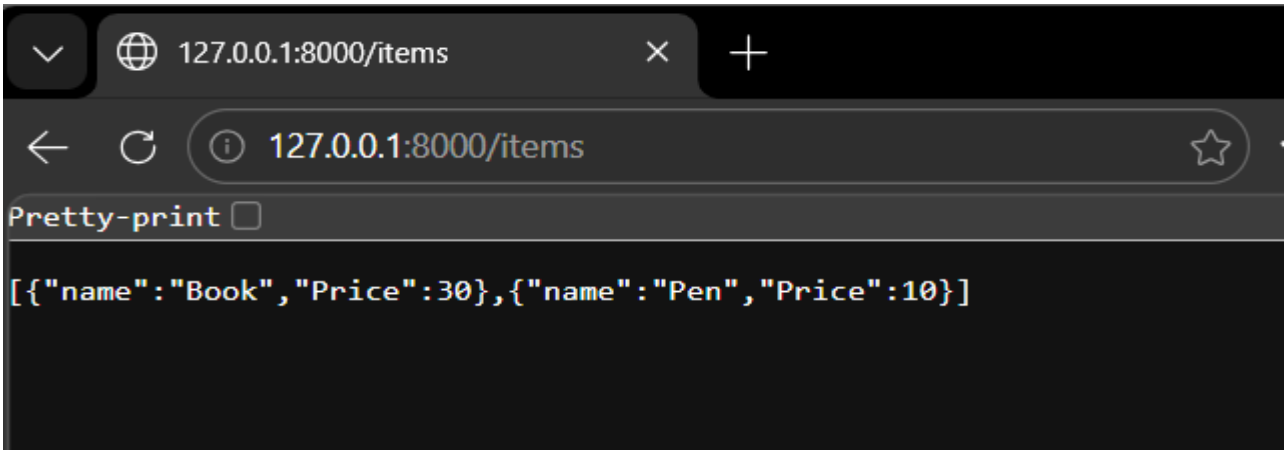
Prompt: Generate an endpoint to update an item using its index.

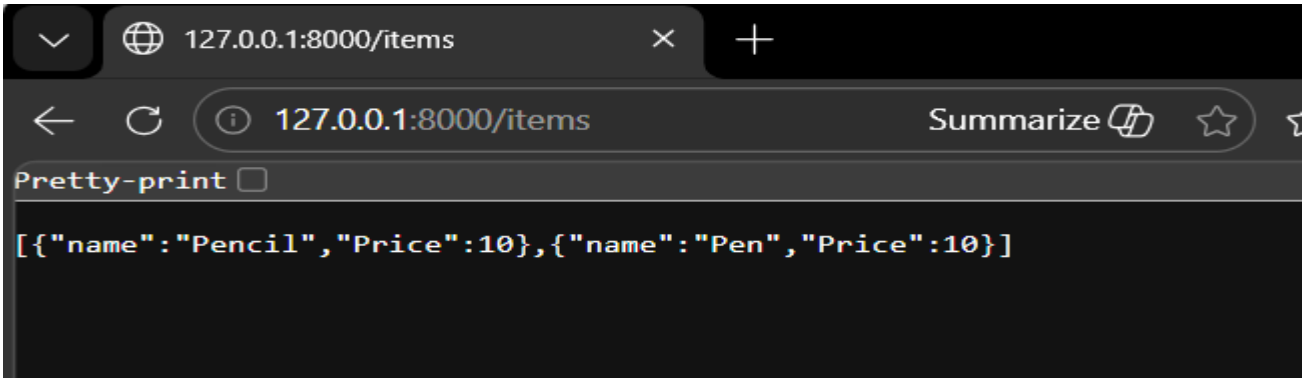
Code:

```python
from fastapi import FastAPI
app = FastAPI()
items = []
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index < 0 or index >= len(items):
        return {"message": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
```

Output:

```
(.venv) PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> python -m uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\BURRA SRUTHI\\OneDrive\\Desktop\\3_2\\AI\\AI_code']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [35956] using StatReload
INFO:     Started server process [20824]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```





Explanation:
PUT: This endpoint allows you to update an existing item in the `items` list using its index. The index is provided as a path parameter, and the new item data is expected to be a dictionary in the request body. If the index is valid, the item at that index is updated with the new data, and a success message along with the updated item is returned. If the index is out of bounds, a message indicating that the item was not found is returned.

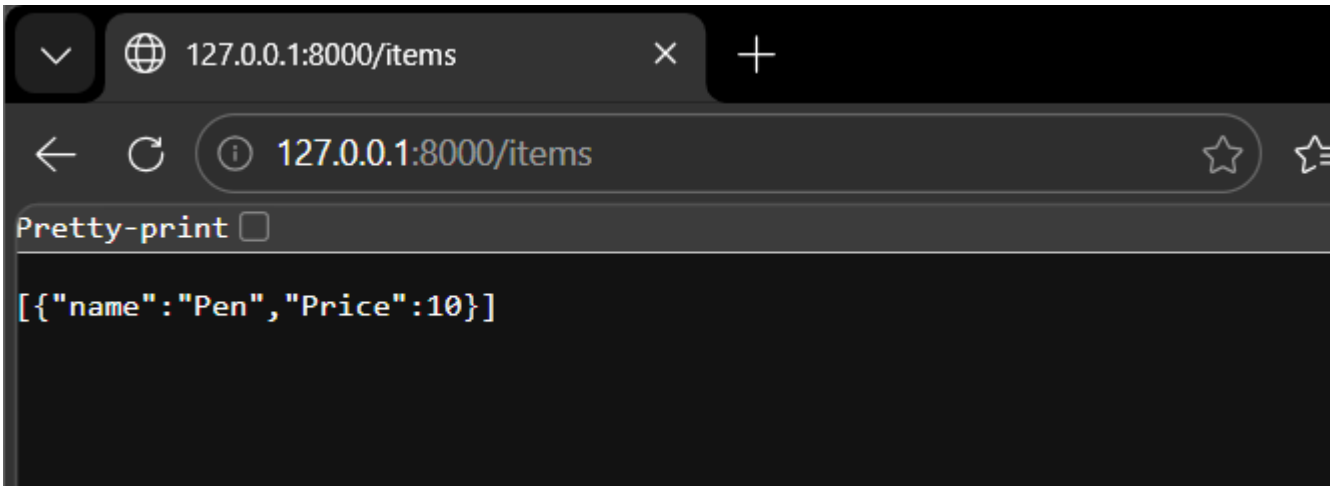
Task 4:

Prompt: Generate an endpoint to delete an item using its index.

Code:

```python
from fastapi import FastAPI
app = FastAPI()
items = []
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
@app.delete("/items/{index}")
def delete_item(index: int):
    if index < 0 or index >= len(items):
        return {"message": "Item not found"}
    deleted_item = items.pop(index)
    return {"message": "Item deleted", "item": deleted_item}
```

Output:



Explanation:
A FastAPI application with endpoints to manage a list of items. The endpoints allow you to retrieve all items, add a new item, update an existing item by its index, and delete an item by its index. The items are stored in a Python list, which serves as temporary storage for the application. Each endpoint returns a JSON response indicating the result of the operation.

Task 5:

Prompt: Add inline comments and docstrings for all endpoints and integrate Swagger / Flask-RESTX documentation.

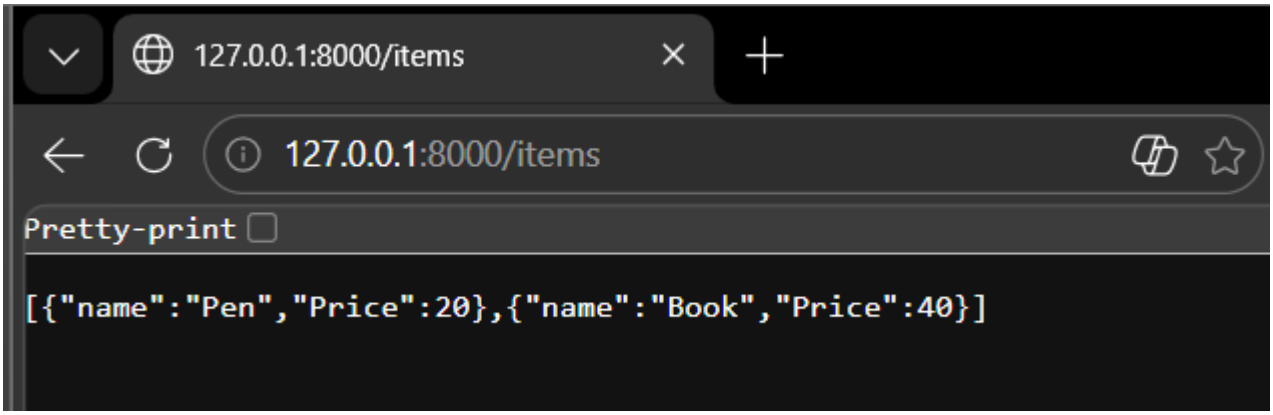
Code:

```python
from fastapi import FastAPI
app = FastAPI()
items = []
@app.get("/items")
def get_items():
    """Endpoint to retrieve all items."""
    return items
@app.post("/items")
def add_item(item: dict):
    """Endpoint to add a new item.
    Args:
        item (dict): The item to be added.
    Returns:
        dict: A message confirming the creation of the item and the item itself.
    """
    items.append(item)
    return {"message": "Item created", "item": item}
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    """Endpoint to update an existing item by its index.
    Args:
        index (int): The index of the item to be updated.
        item (dict): The new item data.
    Returns:
        dict: A message confirming the update of the item and the updated item itself, or an error message if the item is not found.
    """
    if index < 0 or index >= len(items):
        return {"message": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
@app.delete("/items/{index}")
def delete_item(index: int):
    """Endpoint to delete an existing item by its index.
    Args:
        index (int): The index of the item to be deleted.
    Returns:
        dict: A message confirming the deletion of the item and the deleted item itself, or an error message if the item is not found.
    """
    if index < 0 or index >= len(items):
        return {"message": "Item not found"}
    deleted_item = items.pop(index)
    return {"message": "Item deleted", "item": deleted_item}
```

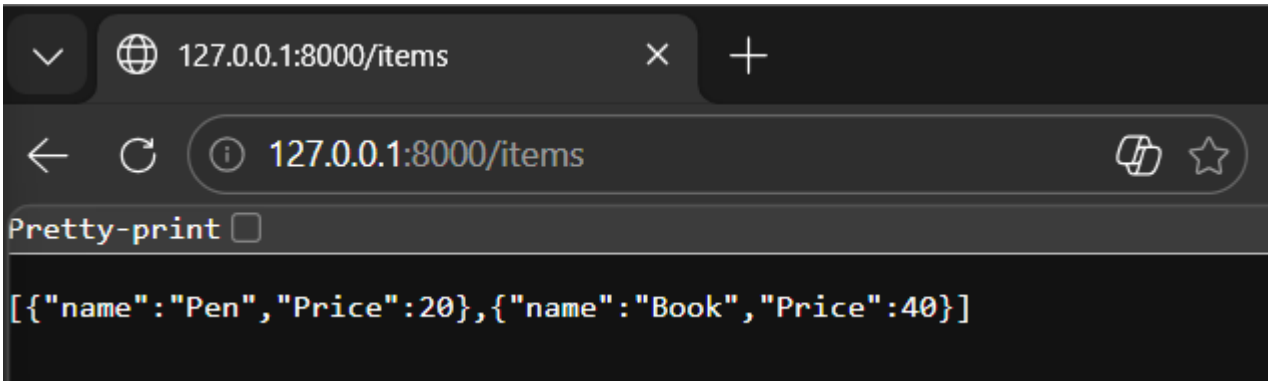
Output:

```
(.venv) PS C:\Users\BURRA SRUTHI\OneDrive\Desktop\3_2\AI\AI_code> python -m uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\BURRA SRUTHI\\OneDrive\\Desktop\\3_
2\\AI\\AI_code']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [5028] using StatReload
INFO:     Started server process [46872]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

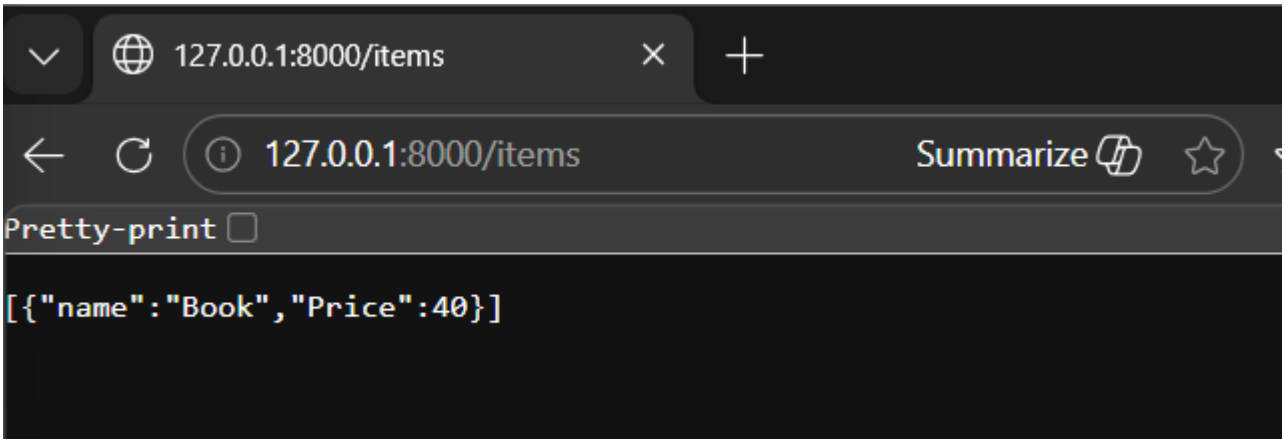
After the Push operation:



After the Put operation:



After the Delete operation:



Explanation:

A FastAPI application is created with a list called `items` to store the items temporarily.
Four endpoints are defined:
1. GET: Retrieves all items in the list.
2. POST: Adds a new item to the list. The item is expected to be a dictionary, and a confirmation message along with the item is returned.
3. PUT: Updates an existing item at the specified index. If the index is valid, the item is updated and a confirmation message is returned. If the index is invalid, an error message is returned.
4. DELETE: Deletes an existing item at the specified index. If the index is valid, the item is deleted and a confirmation message along with the deleted item is returned. If the index is invalid, an error message is returned.